LLVM IR Exploration

Report

Names: Krishnan R, Harikrishna
Roll Numbers: CS24BTECH11036, CS24BTECH11030
Platform: WSL2 (Ubuntu)
Clang Version: Clang version 17.0.6
LLVM Version: 17.0.6 (built from source: llvmorg-17.0.6 with clang and clang-tools-extra, X86 target)

Contents

1 LLVM Directory Layout **2**

2 LLVM IR Study of Core C/C++ Constructs **2**

2.1 C Code in Q2_variables.c (variables) 2

2.2 C Code in Q2_conditionals_loops.c (conditionals and loops) 3

2.3 C Code in Q2_functions.c (functions) 5

2.4 C Code in Q2_casts.c (data types and casts) 6

3 Lowering of Data Structures to LLVM IR **7**

3.1 C Code in Q3_array.c (arrays) 7

3.2 C++ Code in Q3_vector.cpp (std::vector) 7

3.3 C Code in Q3_struct.c (struct) 8

3.4 C Code in Q3_union.c (union) 8

3.5 C++ Code in Q3_class.cpp (class) 9

3.6 Arrays vs vectors 9

3.7 Type of a union 9

3.8 struct vs class 9

4 Floating-Point Representation in LLVM IR **10**

4.1 C Code in Q4_float.c 10

4.2 Representation of floating-point numbers 11

4.3 Operations on floating-point values 11

4.4 Type lowering of floating-point values 11

Introduction and Methodology

This report studies how C/C++ constructs, data structures and floating-point values are represented in LLVM IR, using Clang 17.0.6 at -O0. Every .c/.cpp file in Q2, Q3 and Q4 was compiled with clang -S -emit-llvm -O0 (clang++ for C++), and all outputs can be regenerated with run.sh.

1 LLVM Directory Layout

The LLVM source (llvm-project, release/17.x) has these main directories:

llvm/: the core of LLVM. include/llvm holds the headers, lib/ the implementation (IR, Analysis, Transforms, CodeGen, Target), tools/ programs like llc and opt, and test/ the regression tests.

clang/: the C-family frontend. lib/Lex, lib/Parse, lib/Sema and lib/AST do lexing, parsing, semantic analysis and the AST, and lib/CodeGen converts the AST to LLVM IR.

clang-tools-extra/: extra tools built on Clang, such as clang-tidy and clangd.

Flow used in this report: C/C++ source → clang frontend → LLVM IR (.ll).

2 LLVM IR Study of Core C/C++ Constructs

Four C programs were used: Q2_variables.c (variables), Q2_conditionals_loops.c (conditionals and loops), Q2_functions.c (functions) and Q2_casts.c (data types and casts).

2.1 C Code in Q2_variables.c (variables)

```
#include <stdio.h>
int a = 5; // (i) Global Integer
int main(){
        int b = 10; // (ii) A Local Integer
        int c = 3;  // (ii) A Local Integer
        int d = a + b * c; // (iii) A local Integer initialized to a + b * c
}
```

IR Code Snippets in Q2_variables.ll:

```
@a = dso_local global i32 5, align 4 ; (i) Global Integer a = 5
  %1 = alloca i32, align 4           ; (ii) allocate space for Local Integer b
  %2 = alloca i32, align 4           ; (ii) allocate space for Local Integer c
  %3 = alloca i32, align 4
  store i32 10, ptr %1, align 4      ; (ii) Initialize a Local Integer b = 10
  store i32 3, ptr %2, align 4       ; (ii) Initialize a Local Integer c = 3
  %4 = load i32, ptr @a, align 4     ; load a
  %5 = load i32, ptr %1, align 4     ; load b
  %6 = load i32, ptr %2, align 4     ; load c
  %7 = mul nsw i32 %5, %6            ; b * c
  %8 = add nsw i32 %4, %7            ; a + b * c
  store i32 %8, ptr %3, align 4      ; (iii) Store a + b * c in d
```

Representation of local variables

In LLVM IR, named registers follow the SSA (Static Single Assignment) form, meaning each register is assigned exactly once and does not have a memory address. On the other hand, C variables are real variables stored in memory (stack) which can be done using alloca (which allocates memory on stack to the variable), have an address, can be modified multiple times throughout the program.

Therefore, Clang represents local variables using alloca instructions, which allocate space on the stack, and uses load and store instructions to read and modify their values. This allows LLVM IR to correctly model the mutable nature of variables in C.

2.2 C Code in Q2_conditionals_loops.c (conditionals and loops)

```
#include <stdio.h>
int main(){
        int a = 2, b = 3;
        // (i) If-Else Condition
        if (a == 2){
                a = 2;
        }
        else{
                a = 2;
        }
        // (ii) Switch Case Condition
        switch(a){
                case 1: a = 2; break;
                case 2: a = 2; break;
                default: a = 2; break;
        }
        // (iii) For loop
        for (int i = a; i < 5; i++){
                a++;
        }
        // (iv) While loop
        while (a < 5){
                a++;
        }
        // (v) Do-While loop
        do {
                a++;
        } while ( a < 5);
        return 0;
}
```

IR Code Snippets in Q2_conditionals_loops.ll:

The IR of each construct is shown below. For the loops, the same loop was written in all three forms (for, while and do-while) and their IR was compared.

```
Preds = %x1 %x2... means that the given block can be reached from blocks x1, x2, ...
```

Explanation of each construct:

if-else

```
  %5 = load i32, ptr %2, align 4   ; load a
  %6 = icmp eq i32 %5, 2           ; check: a == 2
  br i1 %6, label %7, label %8     ; if (true) -> go to block 7 else -> go to block 8
7:                                 ; preds = %0, Label 7 starts here
  store i32 2, ptr %2, align 4     ; do: a = 2 then
  br label %9                      ; go to next part (Block 9)
8:                                 ; preds = %0, Label 8 starts here
  store i32 2, ptr %2, align 4     ; do: a = 2 then
  br label %9                      ; go to next part (Block 9)
```

switch

```
9:                                 ; preds = %8, %7, Label 9 starts here
  %10 = load i32, ptr %2, align 4 ; load variable a into ptr 10
```

```
  switch i32 %10, label %13 [     ; switch(ptr 10 = a): else -> go to 13
    i32 1, label %11              ; if a == 1 -> go to 11
    i32 2, label %12              ; else if a == 2 -> go to 12
  ]
11:                               ; preds = %9, Block 11 starts here
  store i32 2, ptr %2, align 4    ; Set a = 2
  br label %14                    ; Go to block 14
12:                               ; preds = %9, Label 12 starts here
  store i32 2, ptr %2, align 4    ; Set a = 2
  br label %14                    ; Go to block 14
13:                               ; preds = %9, Label 13 starts here
  store i32 2, ptr %2, align 4    ; Set a = 2
  br label %14                    ; Go to block 14
```

for:

```
14:                                 ; preds = %13, %12, %11, Label 14 starts here
  %15 = load i32, ptr %2, align 4   ; read value from %2 (variable a), store it in temporary %15
  store i32 %15, ptr %4, align 4    ; store that value into %4
  br label %16                      ; Jump to loop start
16:                                 ; preds = %22, %14, Label 16 starts here
  %17 = load i32, ptr %4, align 4   ; Load variable i into %17
  %18 = icmp slt i32 %17, 5         ; Check if i < 5
  br i1 %18, label %19, label %25 ; If i < 5, branch to loop body, else branch to exit loop
19:                                 ; preds = %16, Label 19, loop body starts here
  %20 = load i32, ptr %2, align 4   ; Load variable a into %20
  %21 = add nsw i32 %20, 1          ; Store a++ into %21
  store i32 %21, ptr %2, align 4    ; Store the new value of a
  br label %22                      ; Branch to label 22
22:                                 ; preds = %19, Label 22 starts here
  %23 = load i32, ptr %4, align 4   ; Load variable i into %23
  %24 = add nsw i32 %23, 1          ; Store i++ into %24
  store i32 %24, ptr %4, align 4    ; Store the new value of i
  br label %16                      ; Branch to label 16
25:                                 ; preds = %16, Label 25 starts here
  br label %26                      ; Branch to label 26
```

while

```
26:                               ; preds = %29, %25, Label 26, while loop starts here
  %27 = load i32, ptr %2, align 4 ; Load the value of variable a into %27
  %28 = icmp slt i32 %27, 5       ; Check if a < 5
  br i1 %28, label %29, label %32 ; Go to loop runner if a < 5, else go to exit loop
29:                               ; preds = %26, Label 29, Loop runner starts here
  %30 = load i32, ptr %2, align 4 ; Load the value of variable a into %30
  %31 = add nsw i32 %30, 1        ; a++
  store i32 %31, ptr %2, align 4  ; Store the new value of a
  br label %26                    ; Branch to label 26 for the next iteration
32:                               ; preds = %26, Label 32, Loop exit starts here
  br label %33                    ; Branch to Label 33
```

do-while

```
33:                         ; preds = %36, %32, Label 33, Do-While loop starts here
  %34 = load i32, ptr %2, align 4 ; Load the value of variable a into %34
  %35 = add nsw i32 %34, 1        ; a++
  store i32 %35, ptr %2, align 4  ; Store the new value of a
  br label %36                    ; Branch to label 36
36:                               ; preds = %33, Label 36 starts here
  %37 = load i32, ptr %2, align 4 ; Load the value of variable a into %37
  %38 = icmp slt i32 %37, 5       ; Check if a < 5
  br i1 %38, label %33, label %39 ; If a < 5, repeat the loop, else exit
39:                               ; preds = %36, Label 39, loop exit starts here
  ret i32 0                       ; int main() returns 0
```

Comparison of the for and while loops

The for and while loops are almost identical in their respective IR's. Both are implemented using a conditional (icmp) and branch (br) block, a loop body, branch instruction to either repeat the loop or exit the loop. The only difference is when we write the for loop in C, we explicitly define the loop iterator variable i, increment steps in the syntax while in the while loop, the compiler takes care of these details.

Position of the condition check in the do-while loop

In both the for and while loops the condition check appears before entering the loop body while in the do-while loop they appear after the loop body finishes one iteration.

2.3 C Code in Q2_functions.c (functions)

```
#include <stdio.h>
int square (int x){
        return x * x;
}
int main(){
        printf("The square of 3 = %d", square(3));
        return 0;
}
```

IR Code Snippets in Q2_functions.ll:

define and declare

The define keyword is present as define dso_local i32 @square(i32 noundef %0) #0 (The definition of the int square(int) function) and as define dso_local i32 @main() #0 (The definition of the main() function)

The declare keyword is present as declare i32 @printf(ptr noundef, ...) #1 (Tells the compiler the function printf() definition is not present in the current file, since it is present in the external library stdio.h included as a header in the file)

The define keyword is used to define functions defined in the given file (like int square (int x) and int main()) itself while the declare keyword is used to declare functions whose definitions are not present in the current file but rather in external files (like printf() in the external library stdio.h declared at the start of the program) but called from the given file.

Passing arguments by value and returning the value

```
define dso_local i32 @square(i32 noundef %0) #0 {
  %2 = alloca i32, align 4         ; Allocate memory to store the argument in %0
  store i32 %0, ptr %2, align 4    ; Receives argument x and stores it in %2
  %3 = load i32, ptr %2, align 4
  %4 = load i32, ptr %2, align 4
  %5 = mul nsw i32 %3, %4
  ret i32 %5         ; Return the value after performing the required calculations
}
%2 = call i32 @square(i32 noundef 3) ; We pass the value 3 into the square() function by value. The
     call instruction is used to receive the return value conveyed back by the function.
```

When main() calls square(3), the value 3 is passed directly to the square() function and received as the parameter %0. It is stored in %2 (allocated using alloca), loaded twice and multiplied to get %5, which is returned using ret i32 %5. In main(), the call instruction gives this returned value in register %2, which is then passed to printf (%3 = call i32 (ptr, ...) @printf(ptr noundef @.str, i32 noundef %2)).

The ret instruction

```
ret i32 %5   ; return result of square
ret i32 0    ; return from main
```

The ret function is used to terminate a function, return the control to the caller while also returning value specified. In the square function ret i32 %5 returns the value x * x while in the main function ret i32 0 returns the value 0 to the OS.

2.4 C Code in Q2_casts.c (data types and casts)

```
#include <stdio.h>
int main(){
        // (i) Assigning Double to an Int (Narrowing)
        int a = 1;
        double b = 2.0;
        a = (int) b;
        // (ii) Assigning Int to a Long (Widening)
        int c = 3;
        long d = 4;
        d = (long) c;
        // (iii) Assigning Long to an Int (Narrowing)
        c = (int) d;
        return 0;
}
```

IR Code Snippets in Q2_casts.ll:

The cast instructions used (fptosi, sext, trunc) and when each one is chosen:

fptosi

```
%6 = load double, ptr %3, align 8
%7 = fptosi double %6 to i32
```

The instruction fptosi (floating point to signed integer) converts a double to an int (narrowing), so the fractional part is lost.

sext

```
%8 = load i32, ptr %4, align 4
%9 = sext i32 %8 to i64
```

The instruction sext (sign extension) widens a 32-bit integer to 64 bits, preserving the sign.

trunc

```
  %10 = load i64, ptr %5, align 8
  %11 = trunc i64 %10 to i32
```

The instruction trunc (truncate) narrows a larger integer (i64, long) to a smaller one (i32, int) by discarding the higher bits, so the value can change if it does not fit.

3 Lowering of Data Structures to LLVM IR

3(a) IR was generated for arrays (3.1), std::vector (3.2), structs (3.3), unions (3.4) and classes (3.5). The questions 3(b), 3(c) and 3(d) are answered in 3.6, 3.7 and 3.8.

3.1 C Code in Q3_array.c (arrays)

```
#include <stdio.h>
int main(){
        int arr1[5];
        for(int i = 0; i < 5; i++){
                arr1[i] = i;
        }
        int d = arr1[2];
        return 0;
}
```

IR Code Snippets in Q3_array.ll:

Array access using getelementptr (GEP)

The getelementptr (GEP) instruction is used to compute the address of an element in an array. GEP does not perform memory access, it only computes addresses.

Ex: %12 = getelementptr inbounds [5 x i32], ptr %2, i64 0, i64 %11

```
[5 x i32]  Array type (5 integers)
ptr %2  Base address of array
i64 0  First index (array itself)
i64 %11  Element index (i)
arr1[i] -> address = base + i * sizeof(int)  GEP Computes this address
```

3.2 C++ Code in Q3_vector.cpp (std::vector)

```
#include <vector>
using namespace std;
int main(){
        vector<int> v;
        v.push_back(1);
        v.push_back(2);
        return v[0];
}
```

IR Code Snippets in Q3_vector.ll (attributes shortened to ...):

```
%"class.std::vector" = type { %"struct.std::_Vector_base" }
%"struct.std::_Vector_base<int, std::allocator<int>>::_Vector_impl_data" = type { ptr, ptr, ptr }
    ; begin, end, end_of_storage
%2 = alloca %"class.std::vector", align 8             ; vector object (3 pointers)
call void @_ZNSt6vectorIiSaIiEEC2Ev(ptr ... %2)         ; constructor
invoke void @_ZNSt6vectorIiSaIiEE9push_backEOi(ptr ... %2, ptr ... %3) to label %7 unwind label %12
       ; v.push_back(1)
%9 = call ptr @_ZNSt6vectorIiSaIiEEixEm(ptr ... %2, i64 noundef 0)   ; v[0]: pointer to element
%10 = load i32, ptr %9, align 4
call void @_ZNSt6vectorIiSaIiEED2Ev(ptr ... %2)         ; destructor
; body of operator[]:
%8 = load ptr, ptr %7, align 8                          ; load the begin pointer
%10 = getelementptr inbounds i32, ptr %8, i64 %9        ; &begin[index]
```

std::vector is a C++ class, not an IR class. So the IR stores it as a struct of three pointers and every operation on it is a function call. push_back uses invoke (function call that tells a function where to go when an exception is thrown) (not call) because it can throw an exception like out of memory error while dynamically allocating, while operator[] loads the begin pointer and uses one getelementptr index to compute the address of the element in the vector at that specified index.

3.3 C Code in Q3_struct.c (struct)

```
#include <stdio.h>
struct Point{
        int x;
        int y;
};
int main(){
        struct Point p;
        p.x = 1;
        p.y = 2;
        return 0;
}
```

IR Code Snippets in Q3_struct.ll:

Type definition

Type definition of struct Point{ int x; int y; }: %struct.Point = type { i32, i32 }

```
%2 = alloca %struct.Point, align 4
store i32 1, ptr %3, align 4
```

The members are accessed with getelementptr using constant indices (0 for x, 1 for y):

```
Struct access:
%3 = getelementptr inbounds %struct.Point, ptr %2, i32 0, i32 0
%4 = getelementptr inbounds %struct.Point, ptr %2, i32 0, i32 1
```

3.4 C Code in Q3_union.c (union)

```
union Data{
        int i;
        float f;
};
int main(){
        union Data d;
        d.i = 1;
        d.f = 1.5f;
        return 0;
}
```

IR Code Snippets in Q3_union.ll:

```
%union.Data = type { i32 }                  ; union type: only one i32
%2 = alloca %union.Data, align 4            ; size 4, alignment 4
store i32 1, ptr %2, align 4                ; d.i = 1
store float 1.500000e+00, ptr %2, align 4   ; d.f = 1.5f  (same address, different type)
```

LLVM IR has no separate union type. A union is represented as a struct with one member (here i32). All the members are accessed using the same address %2, only the type of the load/store changes (i32 for d.i and float for d.f).

3.5 C++ Code in Q3_class.cpp (class)

```
class Point{
public:
        int x;
        int y;
        // Constructor
        Point(){
                x = 0;
                y = 0;
        }
};
int main(){
        Point p;
        return 0;
}
```

IR Code Snippets in Q3_class.ll (attributes shortened):

Type definition and the constructor

Type definition of class Point{ int x; int y; }: %class.Point = type { i32, i32 }

```
call void @_ZN5PointC2Ev(ptr %2)
define void @_ZN5PointC2Ev(ptr %0)
```

Constructors appear as normal functions in LLVM IR. The object pointer (this: %0) is passed as an argument, and the constructor initializes the fields using getelementptr and store instructions. Initializing happens with a constructor call with address of p (Here %2 in the call function).

3.6 Arrays vs vectors

3(b) What is the difference between arrays and vectors in LLVM IR?

An array is a built-in LLVM type ([5 x i32]) of fixed size, allocated directly with alloca and accessed with getelementptr using two indices (0, i). A std::vector is a class, i.e. a struct of three pointers (24 bytes) whose elements are on the heap, accessed only through function calls (push_back, operator[]).

3.7 Type of a union

3(c) How is the type of a union determined in LLVM IR?

Clang uses the member which aligns the most with the rule the that it's type address must be a multiple of some number of bytes (largest byte size on a tie) as the union's type and pads the end if the union is bigger than it. Here i and f are both 4 bytes with alignment 4, so the type is { i32 } (size 4, no padding). Adding char c[8] would give { i32, [4 x i8] } (size 8), since c[8] has alignment 1 but needs 8 bytes.

3.8 struct vs class

3(d) What are the differences between struct and class of C++ once lowered in LLVM IR?

There are no structural differences in the type layout between struct and class: both are { i32, i32 } (%struct.Point in 3.3 and %class.Point in 3.5). The only difference is the constructor of the class, which appears as a normal function in LLVM IR (3.5).

4 Floating-Point Representation in LLVM IR

4.1 C Code in Q4_float.c

```
#include <stdio.h>
int main(){
        float a = 2.3f;
        double b = 5.7;
        b = b + a;               // fpext + fadd
        a = (float) b * a;       // fptrunc + fmul
        int c = (int) (b - a);   // fpext + fsub + fptosi
        if(b < a){               // fpext + fcmp
                double d = c / a;    // sitofp + fdiv + fpext
                a = -a;              // fneg
        }
        return (int) c;
}
```

IR Code Snippets in Q4_float.ll:

```
%1 = alloca i32, align 4                        ; return value of main
%2 = alloca float, align 4                      ; float a
%3 = alloca double, align 8                     ; double b
%4 = alloca i32, align 4                        ; int c
%5 = alloca double, align 8                     ; double d
store i32 0, ptr %1, align 4                    ; return slot = 0
store float 0x4002666660000000, ptr %2, align 4 ; a = 2.3f
store double 5.700000e+00, ptr %3, align 8      ; b = 5.7

; b = b + a
%6 = load double, ptr %3, align 8              ; load b
%7 = load float, ptr %2, align 4               ; load a
%8 = fpext float %7 to double                  ; a: float -> double
%9 = fadd double %6, %8                        ; b + a
store double %9, ptr %3, align 8               ; b = b + a

; a = (float) b * a
%10 = load double, ptr %3, align 8             ; load b
%11 = fptrunc double %10 to float              ; (float) b: double -> float
%12 = load float, ptr %2, align 4              ; load a
%13 = fmul float %11, %12                      ; (float) b * a
store float %13, ptr %2, align 4               ; a = result

; int c = (int) (b - a)
%14 = load double, ptr %3, align 8             ; load b
%15 = load float, ptr %2, align 4              ; load a
%16 = fpext float %15 to double                ; a: float -> double
%17 = fsub double %14, %16                     ; b - a
%18 = fptosi double %17 to i32                 ; (int): double -> int
store i32 %18, ptr %4, align 4                 ; c = result

; if (b < a)
%19 = load double, ptr %3, align 8             ; load b
%20 = load float, ptr %2, align 4              ; load a
%21 = fpext float %20 to double                ; a: float -> double
%22 = fcmp olt double %19, %21                 ; b < a (ordered, less than)
br i1 %22, label %23, label %31                ; true -> block 23, false -> 31

23:                                            ; body of the if
; double d = c / a
%24 = load i32, ptr %4, align 4                ; load c
%25 = sitofp i32 %24 to float                  ; c: int -> float
%26 = load float, ptr %2, align 4              ; load a
```

```
%27 = fdiv float %25, %26                       ; c / a
%28 = fpext float %27 to double                 ; result: float -> double
store double %28, ptr %5, align 8               ; d = result

; a = -a
%29 = load float, ptr %2, align 4              ; load a
%30 = fneg float %29                            ; -a
store float %30, ptr %2, align 4                ; a = -a
br label %31                                    ; leave the if

31:                                             ; after the if
; return (int) c
%32 = load i32, ptr %4, align 4                ; load c (already int, no cast)
ret i32 %32                                     ; return c
```

4.2 Representation of floating-point numbers

LLVM IR has float (32 bit), double (64 bit), and also half, x86_fp80 and fp128. C float and double map to float and double. A constant is written in decimal (5.700000e+00) if the decimal text converts back to exactly the stored value, otherwise in hexadecimal (1.0f/3.0f is 0x3FD5555555555555, the double holding the nearest float to 2.3).

4.3 Operations on floating-point values

Floating point Arithmetic uses fadd, fsub, fmul, fdiv, frem and fneg, and both operands must have the same type. The fcmp operation returns an i1 (integer of size 1 bit which is used to store 0/1 bit values for false/true respectively) and is used by the br (branch instruction which acts like an if else).

4.4 Type lowering of floating-point values

LLVM IR has no implicit conversions, so Clang inserts an explicit instruction for each one: fpext (float to double, in b + a, b < a and the result of c / a), fptrunc (double to float), fptosi (double to int) and sitofp (int to float, in c / a).

Declaration of usage of AI/LLMs

AI tools were used for converting our explanations to LaTeX/PDF, wording changes, checking whether our programs cover the required constructs, verifying results, understanding LLVM IR and clearing doubts. All code was written by us. LLVM was installed and all outputs were generated and checked by us. AI tools were not used to copy.

References

[1] LLVM Project, "LLVM Language Reference Manual", https://llvm.org/docs/LangRef.html

[2] LLVM Project, "Getting Started with the LLVM System" (directory layout, building from source), https://llvm.org/docs/GettingStarted.html

[3] LLVM Project, "The Often Misunderstood GEP Instruction", https://llvm.org/docs/GetElementPtr.html